LAPORAN PRAKTIKUM
PEMROGRAMAN MOBILE
MODUL 3

BUILD A SCROLLABLE LIST

Oleh:

Ashma Salimah NIM. 2410817220015

PROGRAM STUDI TEKNOLOGI INFORMASI

FAKULTAS TEKNIK

UNIVERSITAS LAMBUNG MANGKURAT

APRIL 2026

LEMBAR PENGESAHAN
LAPORAN PRAKTIKUM PEMROGRAMAN MOBILE
MODUL 3

Laporan Praktikum Pemrograman Mobile Modul 3: Build a Scrollable List ini disusun sebagai syarat lulus mata kuliah Praktikum Pemrograman Mobile. Laporan Praktikum ini dikerjakan oleh:

Nama Praktikan : Ashma Salimah
NIM : 2410817220015

Menyetujui, Asisten Praktikum	Mengetahui, Dosen Penanggung Jawab Praktikum
Muhammad Erza Raihan NIM. 2310817210027	Irham Maulani Abdul Gani, S.Kom., M.Kom. NIP. 199710312025061009

DAFTAR ISI

LEMBAR PENGESAHAN 2
DAFTAR ISI 3
DAFTAR GAMBAR 4
DAFTAR TABEL 5
SOAL 1 6
A. Source Code 8
B. Output Program 46
C. Pembahasan 53
SOAL 2 58
A. Jawaban: 58
SOAL 3 60
A. Jawaban 60
TAUTAN GITHUB 62

DAFTAR GAMBAR

Gambar 1. Contoh UI List 7
Gambar 2. Contoh UI Detail 7
Gambar 3. Contoh Penerapan LazyRow/Carousel 8
Gambar 4. Screenshot Awal Aplikasi Compose 46
Gambar 5. Screenshot Tampilan Carousel Compose 46
Gambar 6. Screenshot Tampilan List Compose 47
Gambar 7. Screenshot Tampilan Detail Compose 47
Gambar 8. Screenshot Tampilan Pergantain Bahasa Compose 48
Gambar 9. Screenshot Tampilan Setelah Ganti Bahasa Compose 49
Gambar 10. Screenshot Tampilan Landscape Compose 49
Gambar 11. Screenshot Awal Aplikasi XML 50
Gambar 12. Screenshot Tampilan Carousel XML 50
Gambar 13. Screenshot Tampilan List XML 51
Gambar 14. Screenshot Tampilan Detail XML 51
Gambar 15. Screenshot Tampilan Pergantain Bahasa XML 52
Gambar 16. Screenshot Tampilan Setelah Ganti Bahasa XML 52
Gambar 17. Screenshot Tampilan Landscape XML 53

DAFTAR TABEL

Tabel 1. Source Code MainActivity.kt Compose 8
Tabel 2. Source Code Anime.kt Compose 11
Tabel 3. Source Code AnimeData.kt Compose 11
Tabel 4. Source Code AnimeCarousel.kt Compose 13
Tabel 5. Source Code AnimeList.kt Compose 16
Tabel 6. Source Code DetailScreen.kt Compose 19
Tabel 7. Source Code SettingScreen.kt Compose 21
Tabel 8. Source Code AnimeViewModel.kt Compose 23
Tabel 9. Source Code MainActivity.kt XML 23
Tabel 10. Source Code Anime.kt XML 24
Tabel 11. Source Code AnimeData.kt XML 24
Tabel 12. Source Code HomeFragment.kt XML 26
Tabel 13. Source Code DetailFragment.kt XML 28
Tabel 14. Source Code SettingsFragment.kt XML 29
Tabel 15. Source Code AnimeViewModel.kt XML 31
Tabel 16. Source Code AnimeAdapter.kt XML 32
Tabel 17. Source Code CarouselAdapter.kt XML 33
Tabel 18. Source Code CarouselWrapperAdapter.kt XML 34
Tabel 19. Source Code activity_main.xml 36
Tabel 20. Source Code fragment_home.xml 37
Tabel 21. Source Code fragment_detail.xml 38
Tabel 22. Source Code fragment_setting.xml 40
Tabel 23. Source Code item_anime.xml 41
Tabel 24. Source Code item_carousel.xml 43
Tabel 25. Source Code item_carousel_wrapper.xml 45

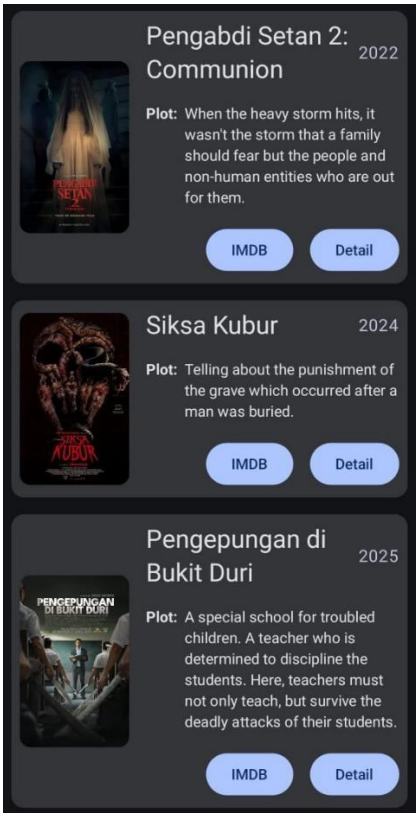
SOAL 1

Buatlah sebuah aplikasi Android menggunakan XML dan Jetpack Compose yang dapat menampilkan list dengan ketentuan berikut:

1. List menggunakan fungsi RecyclerView (XML) dan LazyColumn (Compose)
2. List paling sedikit menampilkan 5 item. Tema item yang ingin ditampilkan bebas
3. Item pada list menampilkan teks dan gambar sesuai dengan contoh di bawah
4. Terdapat 2 button dalam list, dengan fungsi berikut: Button pertama menggunakan intent eksplisit untuk membuka aplikasi atau browser lain
5. Button kedua menggunakan Navigation component untuk membuka laman detail item
6. Sudut item pada list dan gambar di dalam list melengkung atau rounded corner menggunakan Radius
7. Saat orientasi perangkat berubah/dirotasi, baik ke portrait maupun landscape, aplikasi responsif dan dapat menunjukkan list dengan baik. Data di dalam list tidak boleh hilang
8. Aplikasi menggunakan arsitektur *single activity* (satu activity memiliki beberapa fragment)
9. Aplikasi berbasis XML harus menggunakan ViewBinding 9. Aplikasi Mendukung Multi-language (bahasa indonesia dan inggris)
10. Terdapat 1 halaman/screen untuk melakukan pergantian bahasa
11. Gunakan Icon dari library ‘material’ (material,material2,material3) untuk membuat tombol yang mengarahkan ke halaman untuk melakukan pergantian bahasa, dan letakkan di bagian header/heading homescreen utama
12. Diatas Scrollable list, terdapat sebuah “Item Highlight” yang menggunakan Carousel atau LazyRow dengan ketentuan:
    - Item bisa bergeser secara finite atau infinite loop sebanyak jumlah item yang ada pada scrollable-list

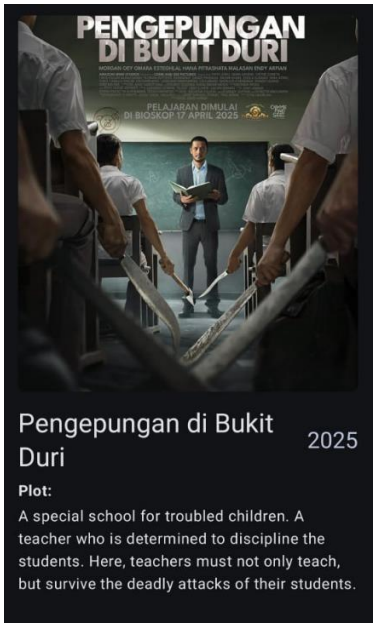
UI item list harus berisi 1 gambar, 2 button (intent eksplisit dan navigasi), dan 2 baris teks dan setiap baris memiliki 2 teks yang berbeda. Diusahakan agar desain UI item list

menyerupai UI berikut:

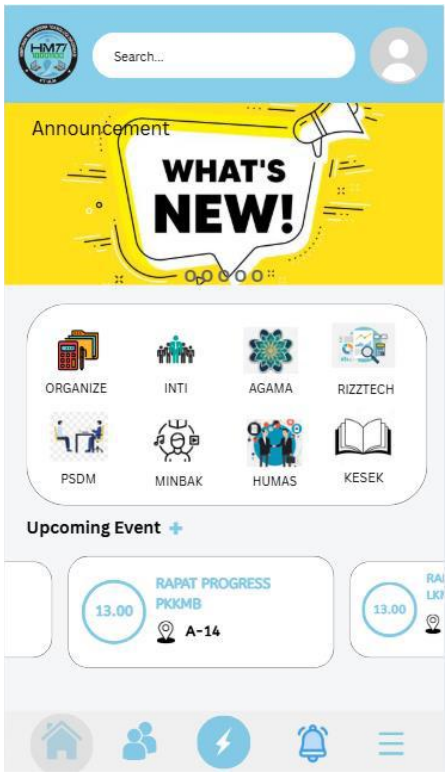


Gambar 1. Contoh UI List

Desain UI laman detail bebas, tetapi diusahakan untuk mengikuti kaidah desain Material Design dan data item ditampilkan penuh di laman detail seperti contoh berikut:



Gambar 2. Contoh UI Detail



Gambar 3. Contoh Penerapan LazyRow/Carousel

Item yang bergeser bisa secara finite (mentok kiri kanan) atau secara infinite loop (misal dari item terakhir '5'kembali ke item awal '1')

A. Source Code

Tabel 1. Source Code MainActivity.kt Compose

No	MainActivity.kt (Compose)
1	`package com.example.composeModul3`
2	
3	`import android.os.Bundle`
4	`import androidx.activity.ComponentActivity`
5	`import androidx.activity.compose.setContent`
6	`import androidx.activity.enableEdgeToEdge`
7	`import androidx.compose.foundation.layout.PaddingValues`
8	`import androidx.compose.foundation.lazy.LazyColumn`
9	`import androidx.compose.material.icons.Icons`
10	`import androidx.compose.material3.ExperimentalMaterial3Api`
11	`import androidx.compose.material3.Icon`
12	`import androidx.compose.material3.IconButton`
13	`import androidx.compose.material3.Scaffold`
14	`import androidx.compose.material3.Text`

```
import androidx.compose.material3.TopAppBar
import androidx.compose.runtime.Composable
import androidx.compose.ui.text.font.FontWeight
import androidx.navigation.NavHostController
import androidx.navigation.compose.NavHost
import androidx.navigation.compose.composable
import
androidx.navigation.compose.rememberNavController
import com.example.composeModul3.data.animeList
import
com.example.composeModul3.ui.components.AnimeList
import com.example.composeModul3.ui.theme.COMPOSETheme
import androidx.compose.foundation.lazy.items
import
com.example.composeModul3.ui.components.AnimeCarousel
import
com.example.composeModul3.ui.screen.DetailsScreen
import
com.example.composeModul3.ui.screen.SettingScreen
import androidx.compose.material.icons.filled.Language
import androidx.compose.ui.res.stringResource
import androidx.lifecycle.viewmodel.compose.viewModel

class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?)
{
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContent {
            COMPOSETheme(darkTheme = true) {
                MyApp()
            }
        }
    }
}

@Composable
fun MyApp() {
    val navController = rememberNavController()
    val viewModel: AnimeViewModel = viewModel()
    NavHost(navController, startDestination = "main") {
        composable("main") { MainScreen(navController)
}
        composable("details/{title}") { backStackEntry
->
            val title =
backStackEntry.arguments?.getString("title") ?: ""
```

```
            DetailsScreen(title = title, navController)
        }
        composable("setting") { SettingScreen(navController) }
    }
}

@OptIn(ExperimentalMaterial3Api::class)
@Composable
fun MainScreen(navController: NavHostController) {
    Scaffold(
        topBar = {
            TopAppBar(
                title = { Text(stringResource(R.string.app_name), fontWeight = FontWeight.Bold) },
                actions = {
                    IconButton(onClick = { navController.navigate("setting") }) {
                        Icon(Icons.Default.Language, contentDescription = "settings")
                    }
                }
            )
        }
    ) { innerPadding ->
        Main(paddingValues = innerPadding, navController = navController)
    }
}

@Composable
fun Main(paddingValues: PaddingValues, navController: NavHostController) {
    LazyColumn(contentPadding = paddingValues) {
        item {
            AnimeCarousel(animeList = animeList)
        }
        items(animeList) { anime ->
            AnimeList(anime = anime, navController = navController)
        }
    }
}
```

Tabel 2. Source Code Anime.kt Compose

No	Anime.kt [model] (Compose)
1	package com.example.composeModul3.model
2	
3	data class Anime(
4	val title: String,
5	val premiered: String,
6	val description: Int,
7	val descLong: Int,
8	val image: Int,
9	val imageBg: Int,
10	val webUrl: String
11	)

Tabel 3. Source Code AnimeData.kt Compose

No	AnimeData.kt [data] (Compose)
1	package com.example.composeModul3.data
2	
3	import com.example.composeModul3.R
4	import com.example.composeModul3.model.Anime
5	
6	val *animeList* = *listOf*(
7	Anime(
8	title = "Jujutsu Kaisen",
9	premiered = "Fall 2020",
10	description = R.string.*desc_jjk*,
11	descLong = R.string.*desc_long_jjk*,
12	image = R.drawable.*jjk*,
13	imageBg = R.drawable.*jjk_bg*,
14	webUrl = "https://www.crunchyroll.com/series/GRDV0019R/jujutsu-kaisen"
15	),
16	Anime(
17	title = "Mashle: Magic and Muscles",
18	premiered = "Spring 2023",
19	description = R.string.*desc_mashle*,
20	descLong = R.string.*desc_long_mashle*,
21	image = R.drawable.*mashle*,
22	imageBg = R.drawable.*mashle_bg*,
23	webUrl = "https://www.crunchyroll.com/series/GDKHZEP8W/mashle-magic-and-muscles"
24	),

```
    Anime(
        title = "Kuroshitsuji",
        premiered = "Fall 2008",
        description = R.string.desc_kuro,
        descLong = R.string.desc_long_kuro,
        image = R.drawable.kuro,
        imageBg = R.drawable.kuro_bg,
        webUrl = "https://www.crunchyroll.com/series/GYQ43P3E6/black-butler"
    ),
    Anime(
        title = "Kamonohashi Ron no Kindan Suiri",
        premiered = "Fall 2023",
        description = R.string.desc_ron,
        descLong = R.string.desc_long_ron,
        image = R.drawable.ron,
        imageBg = R.drawable.ron_bg,
        webUrl = "https://www.crunchyroll.com/series/GNVHKN903/ron-kamonohashis-forbidden-deductions"
    ),
    Anime(
        title = "Kimetsu No Yaiba",
        premiered = "Spring 2019",
        description = R.string.desc_kny,
        descLong = R.string.desc_long_kny,
        image = R.drawable.kny,
        imageBg = R.drawable.kny_bg,
        webUrl = "https://www.crunchyroll.com/series/GY5P48XEY/demon-slayer-kimetsu-no-yaiba"
    ),
    Anime(
        title = "Ganbare! Nakamura-kun!!",
        premiered = "-",
        description = R.string.desc_nakamura,
        descLong = R.string.desc_long_nakamura,
        image = R.drawable.nakamura,
        imageBg = R.drawable.nakamura_bg,
        webUrl = "https://www.crunchyroll.com/series/GT00337506/go-for-it-nakamura-kun"
    ),
    Anime(
```

```
        title = "Boku no Hero Academia",
        premiered = "Spring 2016",
        description = R.string.desc_bnha,
        descLong = R.string.desc_long_bnha,
        image = R.drawable.bnha,
        imageBg = R.drawable.bnha_bg,
        webUrl = "https://www.crunchyroll.com/series/G6NQ5DWZ6/my-hero-academia"
    )
)
```

Tabel 4. Source Code AnimeCarousel.kt Compose

No	AnimeCarousel.kt [ui] (Compose)

```
1  package com.example.composeModul3.ui.components
2
3  import androidx.compose.foundation.Image
4  import androidx.compose.foundation.layout.*
5  import androidx.compose.foundation.pager.HorizontalPager
6  import androidx.compose.foundation.pager.rememberPagerState
7  import androidx.compose.foundation.shape.RoundedCornerShape
8  import androidx.compose.material3.*
9  import androidx.compose.runtime.Composable
10 import androidx.compose.ui.Alignment
11 import androidx.compose.ui.Modifier
12 import androidx.compose.ui.graphics.Color
13 import androidx.compose.ui.layout.ContentScale
14 import androidx.compose.ui.res.painterResource
15 import androidx.compose.ui.res.stringResource
16 import androidx.compose.ui.text.font.FontWeight
17 import androidx.compose.ui.unit.dp
18 import com.example.composeModul3.model.Anime
19 import androidx.compose.ui.res.stringResource
20 import com.example.composeModul3.R
21
22 @Composable
23 fun AnimeCarousel(animeList: List<Anime>) {
24
25     val pagerState = rememberPagerState(
26         initialPage = 0,
27         pageCount = { Int.MAX_VALUE })
```

```

    Column(modifier = Modifier.padding(vertical = 8.dp)) {
        Text(
            text = (stringResource(R.string.featured)),
            style = MaterialTheme.typography.titleMedium,
            fontWeight = FontWeight.Bold,
            modifier = Modifier.padding(horizontal = 16.dp, vertical = 4.dp))

        HorizontalPager(
            state = pagerState,
            contentPadding = PaddingValues(horizontal = 32.dp),
            pageSpacing = 12.dp,
            modifier = Modifier
                .fillMaxWidth()
                .height(220.dp)) {
            page ->
            val anime = animeList[page % animeList.size]
            CarouselItem(anime = anime)
        }

        Row(
            modifier = Modifier
                .fillMaxWidth()
                .padding(top = 8.dp),
            horizontalArrangement = Arrangement.Center) {
            repeat(animeList.size) { index ->
                val isSelected = pagerState.currentPage % animeList.size == index
                Box(
                    modifier = Modifier
                        .padding(horizontal = 4.dp)
                        .size(if (isSelected) 8.dp else 6.dp)
                        .then(
                            Modifier.padding(0.dp)),
                    contentAlignment = Alignment.Center) {
                    Surface(
                        modifier = Modifier.fillMaxSize(),
```

```
                        shape = RoundedCornerShape(50),
                        color = if (isSelected) MaterialTheme.colorScheme.primary
                        else MaterialTheme.colorScheme.onSurface.copy(alpha = 0.3f)) {}
                }
            }
        }
    }
}

@Composable
fun CarouselItem(anime: Anime) {
    Card(
        modifier = Modifier
            .fillMaxWidth()
            .height(200.dp),
        shape = RoundedCornerShape(12.dp)) {
        Box(modifier = Modifier.fillMaxSize()) {
            Image(
                painter = painterResource(id = anime.imageBg),
                contentDescription = anime.title,
                contentScale = ContentScale.Crop,
                modifier = Modifier.fillMaxSize())
            Box(
                modifier = Modifier
                    .fillMaxWidth()
                    .align(Alignment.BottomCenter)
                    .padding(12.dp)) {
                Column {
                    Text(
                        text = anime.title,
                        style = MaterialTheme.typography.bodyMedium,
                        fontWeight = FontWeight.Bold,
                        color = Color.White,
                        maxLines = 2)
                    Text(
                        text = anime.premiered,
                        style = MaterialTheme.typography.labelSmall,
                        color = Color.White)
```

```
                    }
                }
            }
        }
}
```

Tabel 5. Source Code AnimeList.kt Compose

No	AnimeList.kt [ui] (Compose)
1	package com.example.composeModul3.ui.components
2	
3	import android.content.Intent
4	import android.net.Uri
5	import androidx.compose.foundation.Image
6	import androidx.compose.foundation.layout.Arrangement
7	import androidx.compose.foundation.layout.Column
8	import androidx.compose.foundation.layout.Row
9	import androidx.compose.foundation.layout.Spacer
10	import androidx.compose.foundation.layout.fillMaxHeight
11	import androidx.compose.foundation.layout.fillMaxWidth
12	import androidx.compose.foundation.layout.height
13	import androidx.compose.foundation.layout.padding
14	import androidx.compose.foundation.layout.width
15	import androidx.compose.foundation.shape.RoundedCornerShape
16	import androidx.compose.material3.Card
17	import androidx.compose.material3.MaterialTheme
18	import androidx.compose.material3.OutlinedButton
19	import androidx.compose.material3.Text
20	import androidx.compose.runtime.Composable
21	import androidx.compose.ui.Modifier
22	import androidx.compose.ui.draw.clip
23	import androidx.compose.ui.layout.ContentScale
24	import androidx.compose.ui.platform.*LocalContext*
25	import androidx.compose.ui.res.painterResource
26	import androidx.compose.ui.res.stringResource
27	import androidx.compose.ui.text.font.FontWeight
28	import androidx.compose.ui.unit.dp
29	import androidx.navigation.NavHostController
30	import androidx.compose.ui.res.stringResource
31	import com.example.composeModul3.R
32	import com.example.composeModul3.model.Anime
33	
34	@Composable
35	fun AnimeList(anime: Anime, navController:

```
NavHostController) {
    val context = LocalContext.current

    Card(
        modifier = Modifier
        .fillMaxWidth()
        .padding(horizontal = 16.dp, vertical = 8.dp),
        shape = RoundedCornerShape(12.dp)) {
        Row(modifier = Modifier.padding(16.dp)) {
            Image(
                painter = painterResource(id = anime.image),
                contentDescription = "",
                contentScale = ContentScale.Crop,
                modifier = Modifier
                    .fillMaxHeight()
                    .clip(RoundedCornerShape(8.dp)))

            Spacer(modifier = Modifier.width(8.dp))

            Column(modifier = Modifier.fillMaxWidth()) {
                Row(modifier = Modifier.fillMaxWidth(),
                    horizontalArrangement = Arrangement.SpaceBetween)  {
                    Text(
                        text = anime.title,
                        style = MaterialTheme.typography.titleMedium,
                        fontWeight = FontWeight.Bold,
                        modifier = Modifier.weight(1f))
                    Text(
                        text = anime.premiered,
                        style = MaterialTheme.typography.bodySmall)
                }

                Spacer(modifier = Modifier.height(4.dp))

                Text(
                    text = (stringResource(anime.description)),
                    style = MaterialTheme.typography.bodySmall,
```

```
72                      maxLines = 4)
73
74                  Spacer(modifier =
Modifier.height(8.dp))
75
76                  Row(horizontalArrangement =
Arrangement.spacedBy(8.dp)){
77                      OutlinedButton(
78                          onClick = {
79                              val intent =
Intent(Intent.ACTION_VIEW) .apply {
80                                  data =
Uri.parse(anime.webUrl)
81 setPackage("com.crunchyroll.crunchyroid")
82                              }
83 if(intent.resolveActivity(context.packageManager) !=
null){
84 context.startActivity(intent)
85                              } else {
86                                  val fallback =
Intent(Intent.ACTION_VIEW, Uri.parse(anime.webUrl))
87 context.startActivity(fallback)
88                              }
89                          },
90                          shape =
RoundedCornerShape(12.dp)) {

91 Text(stringResource(R.string.watch))
92                      }
93
94                      OutlinedButton(
95                           onClick = {

96 navController.navigate("details/${anime.title}") },
97                          shape =
RoundedCornerShape(12.dp)) {

98 Text(stringResource(R.string.detail))
99                      }
100                 }
101             }
102         }
103     }
104 }
```

Tabel 6. Source Code DetailScreen.kt Compose

No	DetailScreen.kt [screen] (Compose)
1	package com.example.composeModul3.ui.screen
2	
3	import androidx.compose.foundation.Image
4	import androidx.compose.foundation.background
5	import androidx.compose.foundation.layout.*
6	import androidx.compose.foundation.shape.RoundedCornerShape
7	import androidx.compose.material3.*
8	import androidx.compose.runtime.Composable
9	import androidx.compose.ui.Modifier
10	import androidx.compose.ui.graphics.Color
11	import androidx.compose.ui.layout.ContentScale
12	import androidx.compose.ui.res.painterResource
13	import androidx.compose.ui.res.stringResource
14	import androidx.compose.ui.text.font.FontWeight
15	import androidx.compose.ui.unit.dp
16	import com.example.composeModul3.data.*animeList*
17	import androidx.compose.ui.res.stringResource
18	import androidx.navigation.NavHostController
19	import com.example.composeModul3.R
20	
21	@Composable
22	fun DetailsScreen(title: String, navController: NavHostController) {
23	val anime = *animeList*.*find* **{ it**.title == title **}**
24	
25	if (anime == null) {
26	Text("Anime not found")
27	return
28	}
29	
30	Column(modifier = Modifier.*fillMaxSize*().*background*(MaterialTheme.colorScheme.background)) **{**
31	Image(
32	painter = painterResource(id = anime.imageBg),
33	contentDescription = anime.title,
34	contentScale = ContentScale.Crop,
35	modifier = Modifier
36	.*fillMaxWidth*()
37	.*height*(300.*dp*))
38	
39	Column(modifier = Modifier.*padding*(16.*dp*)) **{**
40	Row(

```
41                 modifier = Modifier.fillMaxWidth(),
42                 horizontalArrangement =
Arrangement.SpaceBetween) {
43                 Text(
44                     text = anime.title,
45                     style =
MaterialTheme.typography.headlineSmall,
46                     fontWeight = FontWeight.Bold,
47                     color =
MaterialTheme.colorScheme.onBackground,
48                     modifier = Modifier.weight(1f))
49                 Text(
50                     text = anime.premiered,
51                     style =
MaterialTheme.typography.bodyMedium,
52                     color =
MaterialTheme.colorScheme.onBackground,)
53             }
54
55             Spacer(modifier = Modifier.height(12.dp))
56
57             Text(text =
stringResource(R.string.synopsis),
58                 style =
MaterialTheme.typography.titleMedium,
59                 fontWeight = FontWeight.Bold,
60                 color =
MaterialTheme.colorScheme.onBackground,)
61
62             Spacer(modifier = Modifier.height(4.dp))
63
64             Text(
65                 text =
(stringResource(anime.descLong)),
66                 style =
MaterialTheme.typography.bodyMedium,
67                 color =
MaterialTheme.colorScheme.onBackground)
68
69             Spacer(modifier = Modifier.height(16.dp))
70
71             Button(
72                 onClick = { navController.navigateUp()
},
73                 modifier = Modifier.fillMaxWidth(),
74                 shape = RoundedCornerShape(8.dp),
75                 colors =
```

```
ButtonDefaults.outlinedButtonColors(
                    contentColor = Color(0xFF2D1B45),
                    containerColor = Color(0xFFB39DDB))) {
                Text(text = stringResource(R.string.back))
            }
        }
    }
}
```

Tabel 7. Source Code SettingScreen.kt Compose

No	DetailScreen.kt [screen] (Compose)

```
1  package com.example.composeModul3.ui.screen
2
3  import android.app.LocaleManager
4  import android.content.Context
5  import android.os.Build
6  import android.os.LocaleList
7  import androidx.appcompat.app.AppCompatDelegate
8  import androidx.compose.foundation.layout.*
9  import androidx.compose.foundation.shape.RoundedCornerShape
10 import androidx.compose.material3.*
11 import androidx.compose.runtime.Composable
12 import androidx.compose.ui.Alignment
13 import androidx.compose.ui.Modifier
14 import androidx.compose.ui.graphics.Color
15 import androidx.compose.ui.platform.LocalContext
16 import androidx.compose.ui.res.stringResource
17 import androidx.compose.ui.text.font.FontWeight
18 import androidx.compose.ui.unit.dp
19 import androidx.core.os.LocaleListCompat
20 import com.example.composeModul3.R
21 import androidx.navigation.NavHostController
22
23 @OptIn(ExperimentalMaterial3Api::class)
24 @Composable
25 fun SettingScreen(navController: NavHostController) {
26     val context = LocalContext.current
27
28     Scaffold(
29         topBar = {
30             TopAppBar(
31                 title = {
```

```
32                      Text(
33                          text = stringResource(R.string.settings_title),
34                          fontWeight = FontWeight.Bold)
35                  })
36          }) { innerPadding ->
37          Column(
38              modifier = Modifier
39                  .fillMaxSize()
40                  .padding(innerPadding)
41                  .padding(16.dp),
42              verticalArrangement = Arrangement.spacedBy(12.dp)) {
43              Text(
44                  text = stringResource(R.string.select_language),
45                  style = MaterialTheme.typography.titleMedium,
46                  fontWeight = FontWeight.Bold)
47
48              Button(
49                  onClick = { setLocale(context, "en") },
50                  modifier = Modifier.fillMaxWidth()) {
51                  Text(text = stringResource(R.string.english))
52              }
53
54              Button(
55                  onClick = { setLocale(context, "in")},
56                  modifier = Modifier.fillMaxWidth()) {
57                  Text(text = stringResource(R.string.indonesian))
58              }
59
60              Button(
61                  onClick = { navController.navigateUp() },
62                  modifier = Modifier.fillMaxWidth(),
63                  shape = RoundedCornerShape(8.dp),
64                  colors = ButtonDefaults.outlinedButtonColors(
65                      contentColor = Color(0xFF2D1B45),
66                      containerColor = Color(0xFFB39DDB))) {
67                  Text(text =
```

```
stringResource(R.string.back))
            }
        }
    }
}

fun setLocale(context: Context, language: String) {
    if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.TIRAMISU) {
context.getSystemService(LocaleManager::class.java)
            .applicationLocales = LocaleList.forLanguageTags(language)
    } else {
        AppCompatDelegate.setApplicationLocales(
            LocaleListCompat.forLanguageTags(language)
        )
    }
}
```

Tabel 8. Source Code AnimeViewModel.kt Compose

No	AnimeViewModel.kt (Compose)
1	package com.example.composeModul3
2	
3	import androidx.lifecycle.ViewModel
4	import com.example.composeModul3.data.*animeList* as animeDataList
5	import com.example.composeModul3.model.Anime
6	import kotlinx.coroutines.flow.MutableStateFlow
7	import kotlinx.coroutines.flow.StateFlow
8	
9	class AnimeViewModel : ViewModel() {
10	private val _animeList = *MutableStateFlow*(*animeDataList*)
11	val animeList: StateFlow<List<Anime>> = _animeList
12	}

Tabel 9. Source Code MainActivity.kt XML

No	MainActivity.kt (XML)
1	package com.example.xmlModul3
2	
3	import android.os.Bundle
4	import androidx.appcompat.app.AppCompatActivity
5	import

```
com.example.xmlModul3.databinding.ActivityMainBinding

class MainActivity : AppCompatActivity() {
    private lateinit var binding: ActivityMainBinding

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        binding = ActivityMainBinding.inflate(layoutInflater)
        setContentView(binding.root)
    }
}
```

Tabel 10. Source Code Anime.kt XML

No	Anime.kt [model] (XML)

```
package com.example.animexml.model

data class Anime(
    val title: String,
    val premiered: String,
    val description: Int,
    val descLong: Int,
    val image: Int,
    val imageBg: Int,
    val webUrl: String
)
```

Tabel 11. Source Code AnimeData.kt XML

No	AnimeData.kt [data] (XML)

```
package com.example.xmlModul3.data

import com.example.animexml.model.Anime
import com.example.xmlModul3.R

val animeList = listOf(
    Anime(
        title = "Jujutsu Kaisen",
        premiered = "Fall 2020",
        description = R.string.desc_jjk,
        descLong = R.string.desc_long_jjk,
        image = R.drawable.jjk,
        imageBg = R.drawable.jjk_bg,
        webUrl = "https://www.crunchyroll.com/series/GRDV0019R/jujutsu-kaisen"
```

```
    ),
    Anime(
        title = "Mashle: Magic and Muscles",
        premiered = "Spring 2023",
        description = R.string.desc_mashle,
        descLong = R.string.desc_long_mashle,
        image = R.drawable.mashle,
        imageBg = R.drawable.mashle_bg,
        webUrl = "https://www.crunchyroll.com/series/GDKHZEP8W/mashle-magic-and-muscles"
    ),
    Anime(
        title = "Kuroshitsuji",
        premiered = "Fall 2008",
        description = R.string.desc_kuro,
        descLong = R.string.desc_long_kuro,
        image = R.drawable.kuro,
        imageBg = R.drawable.kuro_bg,
        webUrl = "https://www.crunchyroll.com/series/GYQ43P3E6/black-butler"
    ),
    Anime(
        title = "Kamonohashi Ron no Kindan Suiri",
        premiered = "Fall 2023",
        description = R.string.desc_ron,
        descLong = R.string.desc_long_ron,
        image = R.drawable.ron,
        imageBg = R.drawable.ron_bg,
        webUrl = "https://www.crunchyroll.com/series/GNVHKN903/ron-kamonohashis-forbidden-deductions"
    ),
    Anime(
        title = "Kimetsu No Yaiba",
        premiered = "Spring 2019",
        description = R.string.desc_kny,
        descLong = R.string.desc_long_kny,
        image = R.drawable.kny,
        imageBg = R.drawable.kny_bg,
        webUrl = "https://www.crunchyroll.com/series/GY5P48XEY/demon-slayer-kimetsu-no-yaiba"
    ),
    Anime(
```

```
53          title = "Ganbare! Nakamura-kun!!",
54          premiered = "-",
55          description = R.string.desc_nakamura,
56          descLong = R.string.desc_long_nakamura,
57          image = R.drawable.nakamura,
58          imageBg = R.drawable.nakamura_bg,
59          webUrl =
    "https://www.crunchyroll.com/series/GT00337506/go-for-
    it-nakamura-kun"
60      ),
61      Anime(
62          title = "Boku no Hero Academia",
63          premiered = "Spring 2016",
64          description = R.string.desc_bnha,
65          descLong = R.string.desc_long_bnha,
66          image = R.drawable.bnha,
67          imageBg = R.drawable.bnha_bg,
68          webUrl =
    "https://www.crunchyroll.com/series/G6NQ5DWZ6/my-hero-
    academia"
69      )
70  )
```

Tabel 12. Source Code HomeFragment.kt XML

No	HomeFragment.kt [fragment] (XML)

```
 1  package com.example.xmlModul3.ui.fragment
 2
 3  import android.os.Bundle
 4  import android.view.LayoutInflater
 5  import android.view.View
 6  import android.view.ViewGroup
 7  import androidx.fragment.app.Fragment
 8  import androidx.fragment.app.viewModels
 9  import androidx.navigation.fragment.findNavController
10  import androidx.recyclerview.widget.ConcatAdapter
11  import androidx.recyclerview.widget.LinearLayoutManager
12  import com.example.xmlModul3.AnimeViewModel
13  import com.example.xmlModul3.R
14  import com.example.xmlModul3.adapter.AnimeAdapter
15  import com.example.xmlModul3.adapter.CarouselAdapter
16  import
    com.example.xmlModul3.adapter.CarouselWrapperAdapter
17  import
    com.example.xmlModul3.databinding.FragmentHomeBinding
18
```

```
19 class HomeFragment : Fragment() {
20     private var _binding: FragmentHomeBinding? = null
21     private val binding get() = _binding!!
22     private val viewModel: AnimeViewModel by
   viewModels()
23
24     override fun onCreateView(
25         inflater: LayoutInflater, container: ViewGroup?,
26         savedInstanceState: Bundle?
27     ): View {
28         _binding = FragmentHomeBinding.inflate(inflater,
   container, false)
29         return binding.root
30     }
31
32     override fun onViewCreated(view: View,
   savedInstanceState: Bundle?) {
33         super.onViewCreated(view, savedInstanceState)
34         setupToolbar()
35         setupRecyclerView()
36     }
37
38     private fun setupToolbar() {
39         binding.toolbar.title =
   getString(R.string.app_name)
40         binding.toolbar.inflateMenu(R.menu.menu_home)
41         binding.toolbar.setOnMenuItemClickListener {
   menuItem ->
42             when (menuItem.itemId) {
43                 R.id.action_language -> {

44 findNavController().navigate(R.id.action_home_to_setting
   )
45                     true
46                 }
47                 else -> false
48             }
49         }
50     }
51
52     private fun setupRecyclerView() {
53         val carouselAdapter =
   CarouselAdapter(viewModel.animeList.value)
54
55         val animeAdapter =
   AnimeAdapter(viewModel.animeList.value) { anime ->
```

```
56                val bundle = Bundle().apply {
57                    putString("title", anime.title)
58                }

59 findNavController().navigate(R.id.action_home_to_detail,
   bundle)
60            }
61
62            val concatAdapter = ConcatAdapter(
63                CarouselWrapperAdapter(
64                    carouselAdapter = carouselAdapter,
65                    itemCount =
   viewModel.animeList.value.size,
66                    onPageChanged = { _: Int -> }
67                ),
68                animeAdapter
69            )
70
71            binding.recyclerView.layoutManager =
   LinearLayoutManager(requireContext())
72            binding.recyclerView.adapter = concatAdapter
73        }
74
75        override fun onDestroyView() {
76            super.onDestroyView()
77            _binding = null
78        }
79 }
```

Tabel 13. Source Code DetailFragment.kt XML

No	DetailFragment.kt [fragment] (XML)
1	package com.example.xmlModul3.ui.fragment
2	
3	import android.os.Bundle
4	import android.view.LayoutInflater
5	import android.view.View
6	import android.view.ViewGroup
7	import androidx.fragment.app.Fragment
8	import androidx.navigation.fragment.findNavController
9	import com.bumptech.glide.Glide
10	import com.example.xmlModul3.data.animeList
11	import com.example.xmlModul3.databinding.FragmentDetailBinding
12	
13	class DetailFragment : Fragment() {

```
14      private var _binding: FragmentDetailBinding? = null
15      private val binding get() = _binding!!
16
17      override fun onCreateView(
18          inflater: LayoutInflater, container: ViewGroup?,
19          savedInstanceState: Bundle?
20      ): View {
21          _binding =
    FragmentDetailBinding.inflate(inflater, container,
    false)
22          return binding.root
23      }
24
25      override fun onViewCreated(view: View,
    savedInstanceState: Bundle?) {
26          super.onViewCreated(view, savedInstanceState)
27
28          val title = arguments?.getString("title") ?: ""
29          val anime = animeList.find { it.title == title }
30
31          anime?.let {
32              binding.textTitle.text = it.title
33              binding.textPremiered.text = it.premiered
34              binding.textDescription.text =
    getString(it.descLong)
35
36              Glide.with(this)
37                  .load(it.imageBg)
38                  .centerCrop()
39                  .into(binding.imageBg)
40          }
41
42          binding.buttonBack.setOnClickListener {
43              findNavController().navigateUp()
44          }
45      }
46
47      override fun onDestroyView() {
48          super.onDestroyView()
49          _binding = null
50      }
51  }
```

Tabel 14. Source Code SettingsFragment.kt XML

No	SettingsFragment.kt [fragment] (XML)

```
1  package com.example.xmlModul3.ui.fragment
2
3  import android.app.LocaleManager
4  import android.content.Context
5  import android.os.Build
6  import android.os.Bundle
7  import android.os.LocaleList
8  import android.view.LayoutInflater
9  import android.view.View
10 import android.view.ViewGroup
11 import androidx.appcompat.app.AppCompatDelegate
12 import androidx.core.os.LocaleListCompat
13 import androidx.fragment.app.Fragment
14 import androidx.navigation.fragment.findNavController
15 import
   com.example.xmlModul3.databinding.FragmentSettingBinding
16
17 class SettingFragment : Fragment() {
18     private var _binding: FragmentSettingBinding? = null
19     private val binding get() = _binding!!
20
21     override fun onCreateView(
22         inflater: LayoutInflater, container: ViewGroup?,
23         savedInstanceState: Bundle?
24     ): View {
25         _binding =
   FragmentSettingBinding.inflate(inflater, container,
   false)
26         return binding.root
27     }
28
29     override fun onViewCreated(view: View,
   savedInstanceState: Bundle?) {
30         super.onViewCreated(view, savedInstanceState)
31
32         binding.buttonEnglish.setOnClickListener {
33             setLocale(requireContext(), "en")
34         }
35
36         binding.buttonIndonesian.setOnClickListener {
37             setLocale(requireContext(), "in")
38         }
39
40         binding.buttonBack.setOnClickListener {
41             findNavController().navigateUp()
42         }
```

```
43      }
44
45      private fun setLocale(context: Context, language:
    String) {
46          if (Build.VERSION.SDK_INT >=
    Build.VERSION_CODES.TIRAMISU) {
47              android.util.Log.d("SettingFragment", "Using
    LocaleManager")

48  context.getSystemService(LocaleManager::class.java)
49                  .applicationLocales =
    LocaleList.forLanguageTags(language)
50          } else {
51              android.util.Log.d("SettingFragment", "Using
    AppCompatDelegate")
52              AppCompatDelegate.setApplicationLocales(

53  LocaleListCompat.forLanguageTags(language)
54              )
55          }
56          requireActivity().recreate()
57      }
58
59      override fun onDestroyView() {
60          super.onDestroyView()
61          _binding = null
62      }
63  }
```

Tabel 15. Source Code AnimeViewModel.kt XML

No	**AnimeViewModel.xml**
1	package com.example.xmlModul3
2	
3	import androidx.lifecycle.ViewModel
4	import com.example.animexml.model.Anime
5	import com.example.xmlModul3.data.*animeList* as animeDataList
6	import kotlinx.coroutines.flow.MutableStateFlow
7	import kotlinx.coroutines.flow.StateFlow
8	
9	class AnimeViewModel : ViewModel() {
10	private val _animeList = *MutableStateFlow*(*animeDataList*)
11	val animeList: StateFlow<List<Anime>> = _animeList
12	}

Tabel 16. Source Code AnimeAdapter.kt XML

No	AnimeAdapter.kt [adapter] (XML)
1	package com.example.xmlModul3.adapter
2	
3	import android.content.Intent
4	import android.net.Uri
5	import android.view.LayoutInflater
6	import android.view.ViewGroup
7	import androidx.recyclerview.widget.RecyclerView
8	import com.bumptech.glide.Glide
9	import com.example.animexml.model.Anime
10	import com.example.xmlModul3.databinding.ItemAnimeBinding
11	
12	class AnimeAdapter(
13	private val animeList: List<Anime>,
14	private val onDetailClick: (Anime) -> Unit
15	) : RecyclerView.Adapter<AnimeAdapter.AnimeViewHolder>() {
16	
17	override fun onCreateViewHolder(parent: ViewGroup, viewType: Int): AnimeViewHolder {
18	val binding = ItemAnimeBinding.inflate(
19	LayoutInflater.from(parent.*context*), parent, false
20	)
21	return AnimeViewHolder(binding)
22	}
23	
24	override fun onBindViewHolder(holder: AnimeViewHolder, position: Int) {
25	holder.bind(animeList[position])
26	}
27	
28	override fun getItemCount() = animeList.size
29	
30	inner class AnimeViewHolder(private val binding: ItemAnimeBinding) :
31	RecyclerView.ViewHolder(binding.*root*) {
32	
33	fun bind(anime: Anime) {
34	binding.textTitle.*text* = anime.title
35	binding.textPremiered.*text* = anime.premiered
36	binding.textDescription.*text* =

```
itemView.context.getString(anime.description)

            Glide.with(binding.root)
                .load(anime.image)
                .centerCrop()
                .into(binding.imageAnime)

            binding.buttonWatch.setOnClickListener {
                val intent = Intent(Intent.ACTION_VIEW).apply {
                    data = Uri.parse(anime.webUrl)
setPackage("com.crunchyroll.crunchyroid")
                }
                if (intent.resolveActivity(itemView.context.packageManager) != null) {
itemView.context.startActivity(intent)
                } else {
                    val fallback = Intent(Intent.ACTION_VIEW, Uri.parse(anime.webUrl))
itemView.context.startActivity(fallback)
                }
            }

            binding.buttonDetail.setOnClickListener {
                onDetailClick(anime)
            }
        }
    }
}
```

Tabel 17. Source Code CarouselAdapter.kt XML

No	CarouselAdapter.kt (XML)
1	package com.example.xmlModul3.adapter
2	
3	import android.view.LayoutInflater
4	import android.view.ViewGroup
5	import androidx.recyclerview.widget.RecyclerView
6	import com.bumptech.glide.Glide
7	import com.example.animexml.model.Anime
8	import com.example.xmlModul3.databinding.ItemCarouselBinding

```
 9
10 class CarouselAdapter(private val animeList:
   List<Anime>) :

11 RecyclerView.Adapter<CarouselAdapter.CarouselViewHolder>
   () {
12
13     override fun getItemCount() = Int.MAX_VALUE
14
15     override fun onCreateViewHolder(parent: ViewGroup,
   viewType: Int): CarouselViewHolder {
16         val binding = ItemCarouselBinding.inflate(
17             LayoutInflater.from(parent.context), parent,
   false
18         )
19         return CarouselViewHolder(binding)
20     }
21
22     override fun onBindViewHolder(holder:
   CarouselViewHolder, position: Int) {
23         holder.bind(animeList[position %
   animeList.size])
24     }
25
26     inner class CarouselViewHolder(private val binding:
   ItemCarouselBinding) :
27         RecyclerView.ViewHolder(binding.root) {
28         fun bind(anime: Anime) {
29             binding.textCarouselTitle.text = anime.title
30             binding.textCarouselPremiered.text =
   anime.premiered
31             Glide.with(binding.root)
32                 .load(anime.imageBg)
33                 .centerCrop()
34                 .into(binding.imageCarousel)
35         }
36     }
37 }
```

Tabel 18. Source Code CarouselWrapperAdapter.kt XML

No	**CarouselWrapperAdapter.kt [adapter] (XML)**
1	`package com.example.xmlModul3.adapter`
2	
3	`import android.view.LayoutInflater`
4	`import android.view.ViewGroup`
5	`import android.widget.ImageView`

```
 6 import androidx.recyclerview.widget.RecyclerView
 7 import androidx.viewpager2.widget.ViewPager2
 8 import com.example.xmlModul3.R
 9 import
   com.example.xmlModul3.databinding.ItemCarouselWrapperBin
   ding
10
11 class CarouselWrapperAdapter(
12     private val carouselAdapter: CarouselAdapter,
13     private val itemCount: Int,
14     private val onPageChanged: (Int) -> Unit
15 ) :
   RecyclerView.Adapter<CarouselWrapperAdapter.ViewHolder>(
   ) {
16
17     override fun onCreateViewHolder(parent: ViewGroup,
   viewType: Int): ViewHolder {
18         val binding =
   ItemCarouselWrapperBinding.inflate(
19             LayoutInflater.from(parent.context), parent,
   false
20         )
21         return ViewHolder(binding)
22     }
23
24     override fun onBindViewHolder(holder: ViewHolder,
   position: Int) {
25         holder.bind()
26     }
27
28     override fun getItemCount() = 1
29
30     inner class ViewHolder(private val binding:
   ItemCarouselWrapperBinding) :
31         RecyclerView.ViewHolder(binding.root) {
32
33         fun bind() {
34             binding.viewPager.adapter = carouselAdapter
35             setupDots()
36


37 binding.viewPager.registerOnPageChangeCallback(object :
38                 ViewPager2.OnPageChangeCallback() {
39                 override fun onPageSelected(position:
   Int) {
40                     onPageChanged(position)
```

```
41                      updateDots(position % itemCount)
42                  }
43              })
44          }
45
46          private fun setupDots() {
47              binding.dotIndicator.removeAllViews()
48              repeat(itemCount) { index ->
49                  val dot =
    ImageView(binding.root.context).apply {
50                      setImageResource(
51                          if (index == 0)
    R.drawable.dot_selected
52                          else R.drawable.dot_unselected
53                      )
54                      val params =
    ViewGroup.MarginLayoutParams(16, 16)
55                      params.setMargins(8, 0, 8, 0)
56                      layoutParams = params
57                  }
58                  binding.dotIndicator.addView(dot)
59              }
60          }
61
62          private fun updateDots(selectedIndex: Int) {
63              for (i in 0 until
    binding.dotIndicator.childCount) {
64                  val dot =
    binding.dotIndicator.getChildAt(i) as ImageView
65                  dot.setImageResource(
66                      if (i == selectedIndex)
    R.drawable.dot_selected
67                      else R.drawable.dot_unselected
68                  )
69              }
70          }
71      }
72  }
```

Tabel 19. Source Code activity_main.xml

No	**activity_main.xml**
1	`<?xml version="1.0" encoding="utf-8"?>`
2	`<androidx.constraintlayout.widget.ConstraintLayout`
3	`xmlns:android="http://schemas.android.com/apk/res/androi`

```
d"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:fitsSystemWindows="true">

    <androidx.fragment.app.FragmentContainerView
        android:id="@+id/nav_host_fragment"

android:name="androidx.navigation.fragment.NavHostFragme
nt"
        android:layout_width="match_parent"
        android:layout_height="match_parent"
        app:defaultNavHost="true"
        app:navGraph="@navigation/nav_graph"
        app:layout_constraintTop_toTopOf="parent"

app:layout_constraintBottom_toBottomOf="parent"/>

</androidx.constraintlayout.widget.ConstraintLayout>
```

Tabel 20. Source Code fragment_home.xml

No	fragment_home.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
xmlns:android="http://schemas.android.com/apk/res/androi
d"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical">

    <androidx.appcompat.widget.Toolbar
        android:id="@+id/toolbar"
        android:layout_width="match_parent"
        android:layout_height="?attr/actionBarSize"
        android:background="?attr/colorSurface"
        android:elevation="4dp"/>

    <LinearLayout
        android:id="@+id/dotIndicator"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:gravity="center"
        android:orientation="horizontal"
        android:padding="8dp"
        android:visibility="gone"/>
```

```
22
23     <androidx.recyclerview.widget.RecyclerView
24         android:id="@+id/recyclerView"
25         android:layout_width="match_parent"
26         android:layout_height="0dp"
27         android:layout_weight="1"
28         android:padding="8dp"/>
29
30 </LinearLayout>
```

Tabel 21. Source Code fragment_detail.xml

No	fragment_detail.xml
1	`<?xml version="1.0" encoding="utf-8"?>`
2	`<ScrollView xmlns:android="http://schemas.android.com/apk/res/andr oid"`
3	`    android:layout_width="match_parent"`
4	`    android:layout_height="match_parent">`
5	
6	`    <LinearLayout`
7	`        android:layout_width="match_parent"`
8	`        android:layout_height="wrap_content"`
9	`        android:orientation="vertical">`
10	
11	`        <ImageView`
12	`            android:id="@+id/imageBg"`
13	`            android:layout_width="match_parent"`
14	`            android:layout_height="300dp"`
15	`            android:scaleType="centerCrop" />`
16	
17	`        <LinearLayout`
18	`            android:layout_width="match_parent"`
19	`            android:layout_height="wrap_content"`
20	`            android:orientation="vertical"`
21	`            android:padding="16dp">`
22	
23	`            <LinearLayout`
24	`                android:layout_width="match_parent"`
25	`                android:layout_height="wrap_content"`
26	`                android:orientation="horizontal">`
27	
28	`                <TextView`
29	`                    android:id="@+id/textTitle"`
30	`                    android:layout_width="0dp"`
31	`android:layout_height="wrap_content"`

```
32                      android:layout_weight="1"

33 android:textAppearance="?attr/textAppearanceHeadlineSm
   all"
34                      android:textStyle="bold"/>
35
36                  <TextView
37                      android:id="@+id/textPremiered"

38 android:layout_width="wrap_content"

39 android:layout_height="wrap_content"

40 android:textAppearance="?attr/textAppearanceBodyMedium
   "/>
41
42              </LinearLayout>
43
44              <View
45                  android:layout_width="match_parent"
46                  android:layout_height="12dp"/>
47
48              <TextView
49                  android:layout_width="wrap_content"
50                  android:layout_height="wrap_content"
51                  android:text="@string/synopsis"

52 android:textAppearance="?attr/textAppearanceTitleMediu
   m"
53                  android:textStyle="bold"/>
54
55              <View
56                  android:layout_width="match_parent"
57                  android:layout_height="4dp"/>
58
59              <TextView
60                  android:id="@+id/textDescription"
61                  android:layout_width="match_parent"
62                  android:layout_height="wrap_content"

63 android:textAppearance="?attr/textAppearanceBodyMedium
   "/>
64
65              <View
66                  android:layout_width="match_parent"
67                  android:layout_height="12dp"/>
68
```

```
69              <Button
70                  android:id="@+id/buttonBack"
71                  android:layout_width="match_parent"
72                  android:layout_height="wrap_content"
73                  android:text="@string/back"/>
74
75          </LinearLayout>
76      </LinearLayout>
77  </ScrollView>
```

Tabel 22. Source Code fragment_setting.xml

No | **fragment_setting.xml**

```
1   <?xml version="1.0" encoding="utf-8"?>
2   <LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/androi
    d"
3       android:layout_width="match_parent"
4       android:layout_height="match_parent"
5       android:orientation="vertical"
6       android:padding="16dp">
7
8       <androidx.appcompat.widget.Toolbar
9           android:id="@+id/toolbar"
10          android:layout_width="match_parent"
11          android:layout_height="?attr/actionBarSize"
12          android:background="?attr/colorSurface"/>
13
14      <TextView
15          android:layout_width="wrap_content"
16          android:layout_height="wrap_content"
17          android:text="@string/select_language"

18  android:textAppearance="?attr/textAppearanceTitleMedium"
19          android:textStyle="bold"
20          android:paddingBottom="12dp"/>
21
22      <Button
23          android:id="@+id/buttonEnglish"
24          android:layout_width="match_parent"
25          android:layout_height="wrap_content"
26          android:text="@string/english"/>
27
28      <Button
29          android:id="@+id/buttonIndonesian"
```

```
30          android:layout_width="match_parent"
31          android:layout_height="wrap_content"
32          android:text="@string/indonesian"
33          android:layout_marginTop="8dp"/>
34
35      <Button
36          android:id="@+id/buttonBack"
37          android:layout_width="match_parent"
38          android:layout_height="wrap_content"
39          android:text="@string/back"
40          android:layout_marginTop="8dp"/>
41
42  </LinearLayout>
```

Tabel 23. Source Code item_anime.xml

No	item_anime.xml

```
1   <?xml version="1.0" encoding="utf-8"?>
2   <com.google.android.material.card.MaterialCardView

3   xmlns:android="http://schemas.android.com/apk/res/andr
    oid"
4       xmlns:app="http://schemas.android.com/apk/res-
    auto"
5       android:layout_width="match_parent"
6       android:layout_height="wrap_content"
7       android:layout_margin="8dp"
8       app:cardCornerRadius="12dp"
9       app:cardElevation="4dp">
10
11      <LinearLayout
12          android:layout_width="match_parent"
13          android:layout_height="wrap_content"
14          android:orientation="horizontal"
15          android:padding="12dp">
16
17          <ImageView
18              android:id="@+id/imageAnime"
19              android:layout_width="100dp"
20              android:layout_height="match_parent"
21              android:scaleType="centerCrop"/>
22
23          <LinearLayout
24              android:layout_width="match_parent"
25              android:layout_height="wrap_content"
26              android:orientation="vertical"
```

```
27              android:layout_marginStart="12dp">
28
29              <LinearLayout
30                  android:layout_width="match_parent"
31                  android:layout_height="wrap_content"
32                  android:orientation="horizontal">
33
34                  <TextView
35                      android:id="@+id/textTitle"
36                      android:layout_width="0dp"

37  android:layout_height="wrap_content"
38                      android:layout_weight="1"
39                      android:textStyle="bold"

40  android:textAppearance="?attr/textAppearanceTitleMediu
    m"/>
41
42                  <TextView
43                      android:id="@+id/textPremiered"

44  android:layout_width="wrap_content"

45  android:layout_height="wrap_content"

46  android:textAppearance="?attr/textAppearanceBodySmall"
    />
47
48              </LinearLayout>
49
50              <TextView
51                  android:id="@+id/textDescription"
52                  android:layout_width="match_parent"
53                  android:layout_height="wrap_content"
54                  android:maxLines="3"
55                  android:layout_marginTop="4dp"

56  android:textAppearance="?attr/textAppearanceBodySmall"
    />
57
58              <LinearLayout
59                  android:layout_width="match_parent"
60                  android:layout_height="wrap_content"
61                  android:orientation="horizontal"
62                  android:layout_marginTop="8dp">
63
```

```
<com.google.android.material.button.MaterialButton
                    android:id="@+id/buttonWatch"

android:layout_width="wrap_content"

android:layout_height="wrap_content"
                    android:text="@string/watch"
                    app:cornerRadius="12dp"

style="@style/Widget.Material3.Button.OutlinedButton"/
>

<com.google.android.material.button.MaterialButton
                    android:id="@+id/buttonDetail"

android:layout_width="wrap_content"

android:layout_height="wrap_content"
                    android:text="@string/detail"
                    android:layout_marginStart="8dp"
                    app:cornerRadius="12dp"

style="@style/Widget.Material3.Button.OutlinedButton"/
>

            </LinearLayout>
        </LinearLayout>
    </LinearLayout>

</com.google.android.material.card.MaterialCardView>
```

Tabel 24. Source Code item_carousel.xml

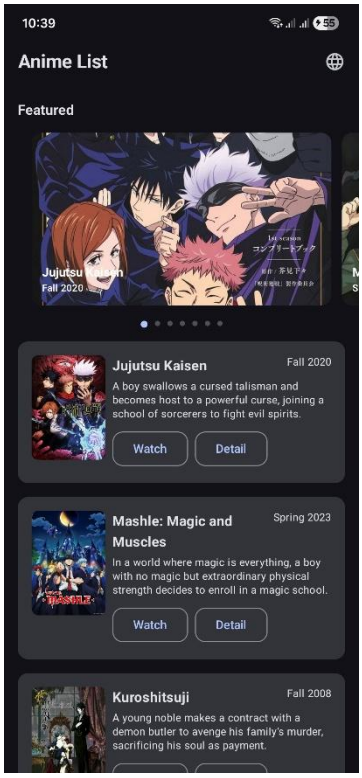
No	item_carousel.xml
1	<?xml version="1.0" encoding="utf-8"?>
2	<com.google.android.material.card.MaterialCardView
3	xmlns:android="http://schemas.android.com/apk/res/andr oid"
4	xmlns:app="http://schemas.android.com/apk/res-auto"
5	android:layout_width="match_parent"
6	android:layout_height="match_parent"
7	android:layout_margin="8dp"
8	app:cardCornerRadius="12dp">

```
 9
10     <FrameLayout
11         android:layout_width="match_parent"
12         android:layout_height="match_parent">
13
14         <ImageView
15             android:id="@+id/imageCarousel"
16             android:layout_width="match_parent"
17             android:layout_height="match_parent"
18             android:scaleType="centerCrop"/>
19
20         <LinearLayout
21             android:layout_width="match_parent"
22             android:layout_height="wrap_content"
23             android:layout_gravity="bottom"
24             android:orientation="vertical"
25             android:padding="12dp"
26             android:background="#80000000">
27
28             <TextView
29                 android:id="@+id/textCarouselTitle"
30                 android:layout_width="match_parent"
31                 android:layout_height="wrap_content"

32 android:textColor="@android:color/white"
33                 android:textStyle="bold"
34                 android:maxLines="2"

35 android:textAppearance="?attr/textAppearanceBodyMedium
   "/>
36
37             <TextView

38 android:id="@+id/textCarouselPremiered"
39                 android:layout_width="match_parent"
40                 android:layout_height="wrap_content"

41 android:textColor="@android:color/white"

42 android:textAppearance="?attr/textAppearanceLabelSmall
   "/>
43
44         </LinearLayout>
45     </FrameLayout>
46
47 </com.google.android.material.card.MaterialCardView>
```

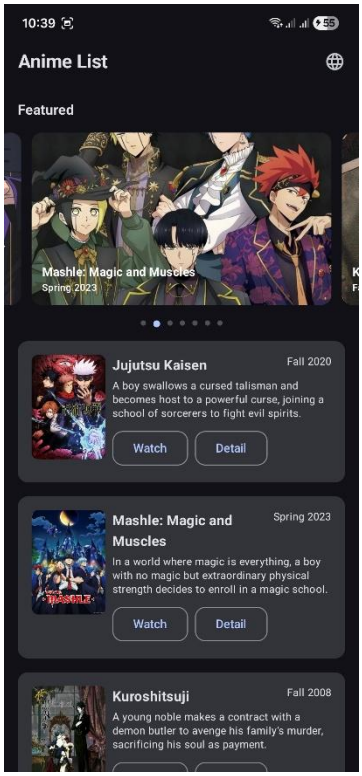
Tabel 25. Source Code item_carousel_wrapper.xml

No	item_carousel_wrapper.xml
1	<?xml version="1.0" encoding="utf-8"?>
2	<LinearLayout
3	xmlns:android="http://schemas.android.com/apk/res/andr oid"
4	android:layout_width="match_parent" android:layout_height="wrap_content"
5	android:orientation="vertical">
6	
7	<androidx.viewpager2.widget.ViewPager2
8	android:id="@+id/viewPager"
9	android:layout_width="match_parent"
10	android:layout_height="220dp"/>
11	
12	<LinearLayout
13	android:id="@+id/dotIndicator"
14	android:layout_width="match_parent"
15	android:layout_height="wrap_content"
16	android:gravity="center"
17	android:orientation="horizontal"
18	android:padding="8dp"/>
19	
20	</LinearLayout>

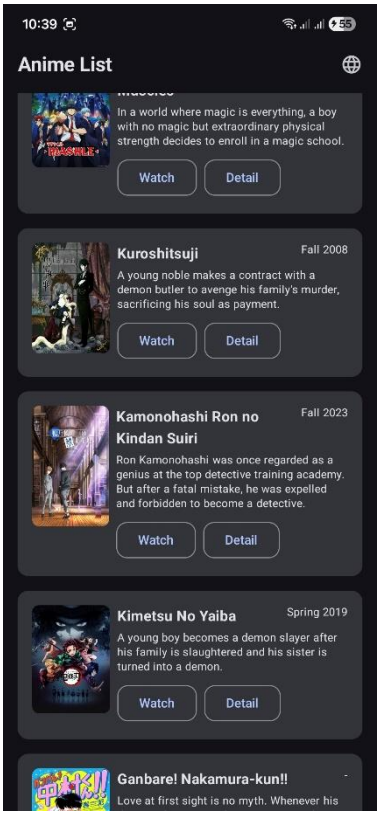
B. Output Program



Gambar 4. Screenshot Awal Aplikasi Compose

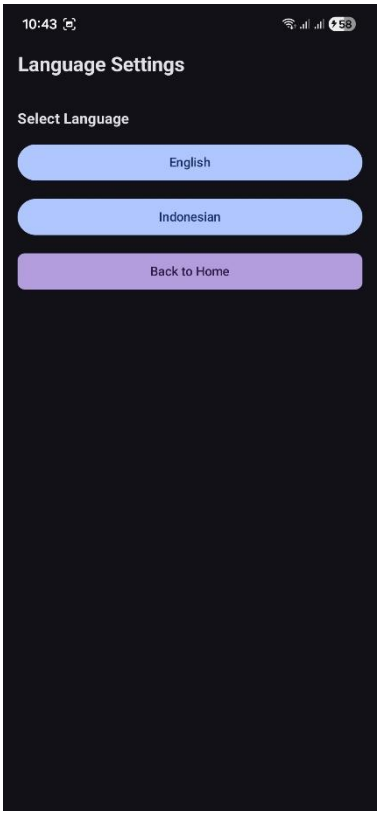


Gambar 5. Screenshot Tampilan Carousel Compose

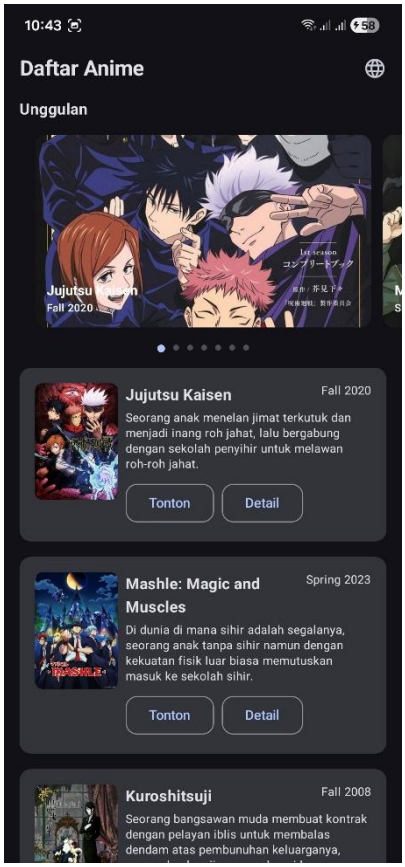


Gambar 6. Screenshot Tampilan List Compose

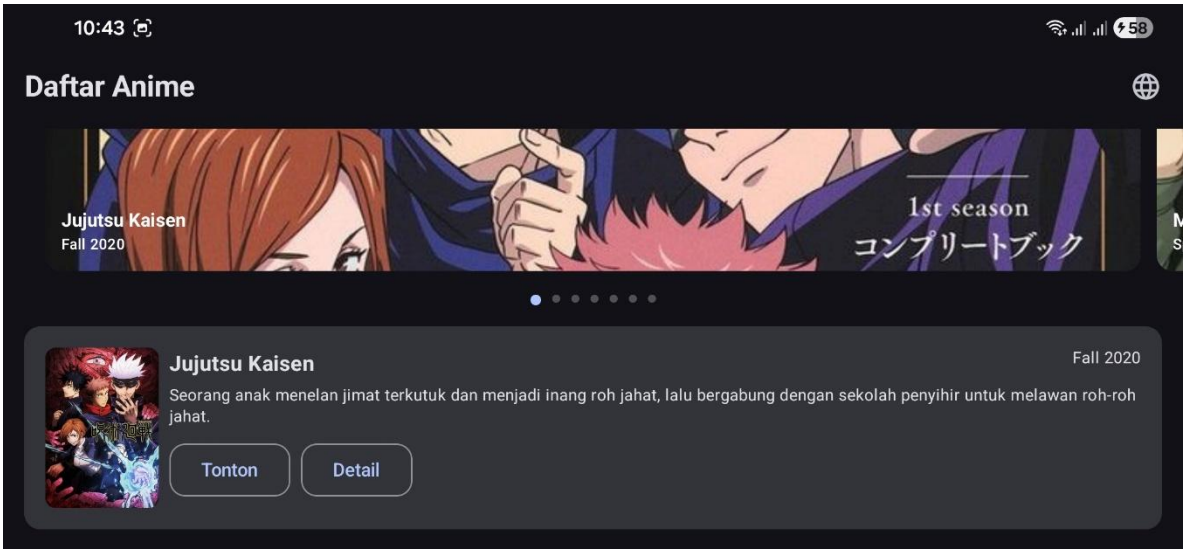
Gambar 7. Screenshot Tampilan Detail Compose



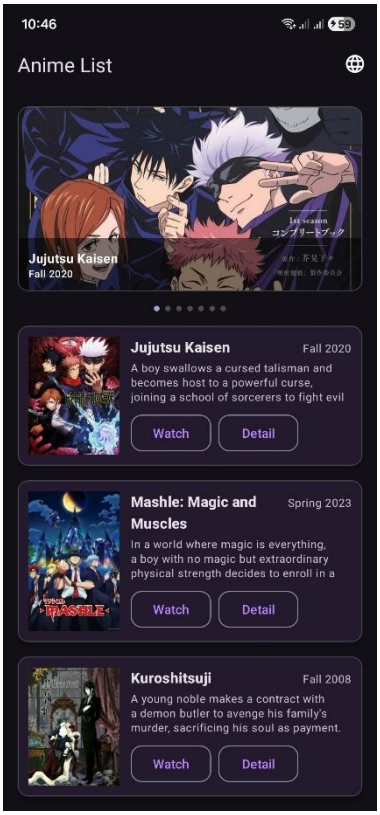
Gambar 8. Screenshot Tampilan Pergantain Bahasa Compose



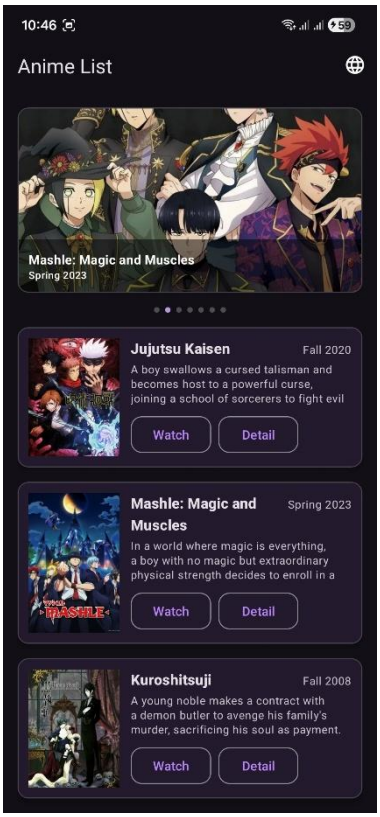
Gambar 9. Screenshot Tampilan Setelah Ganti Bahasa Compose



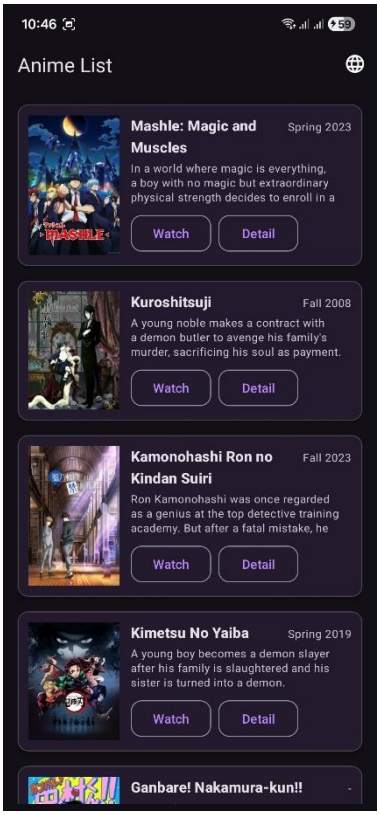
Gambar 10. Screenshot Tampilan Landscape Compose



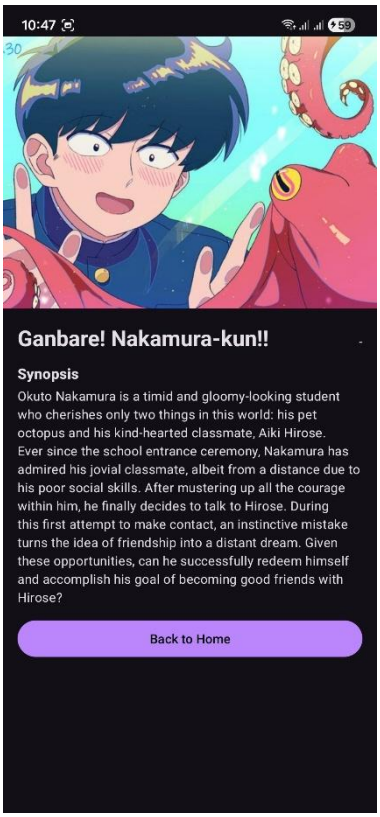
Gambar 11. Screenshot Awal Aplikasi XML



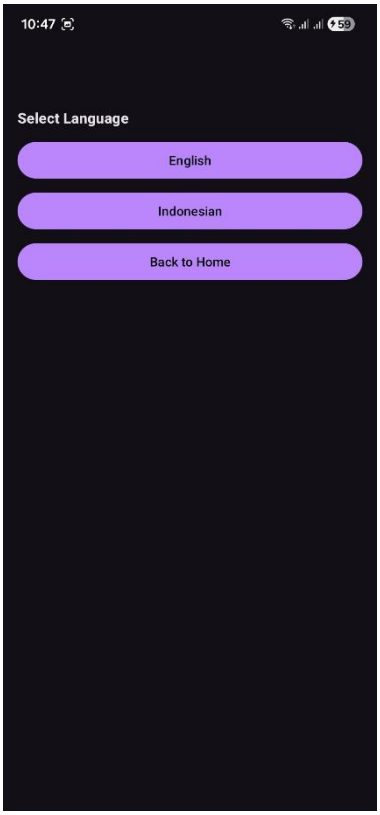
Gambar 12. Screenshot Tampilan Carousel XML



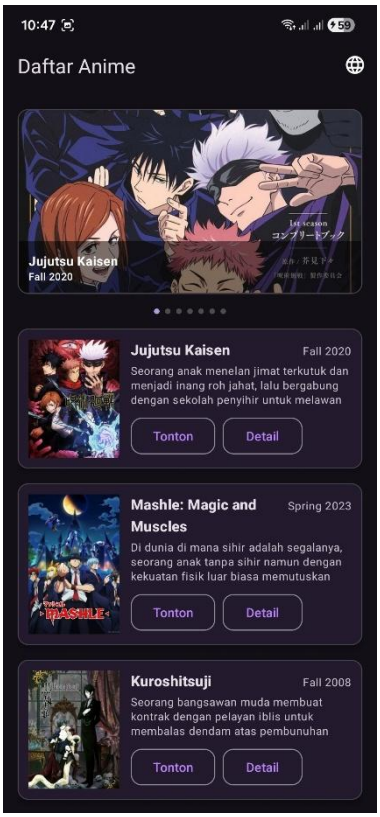
Gambar 13. Screenshot Tampilan List XML



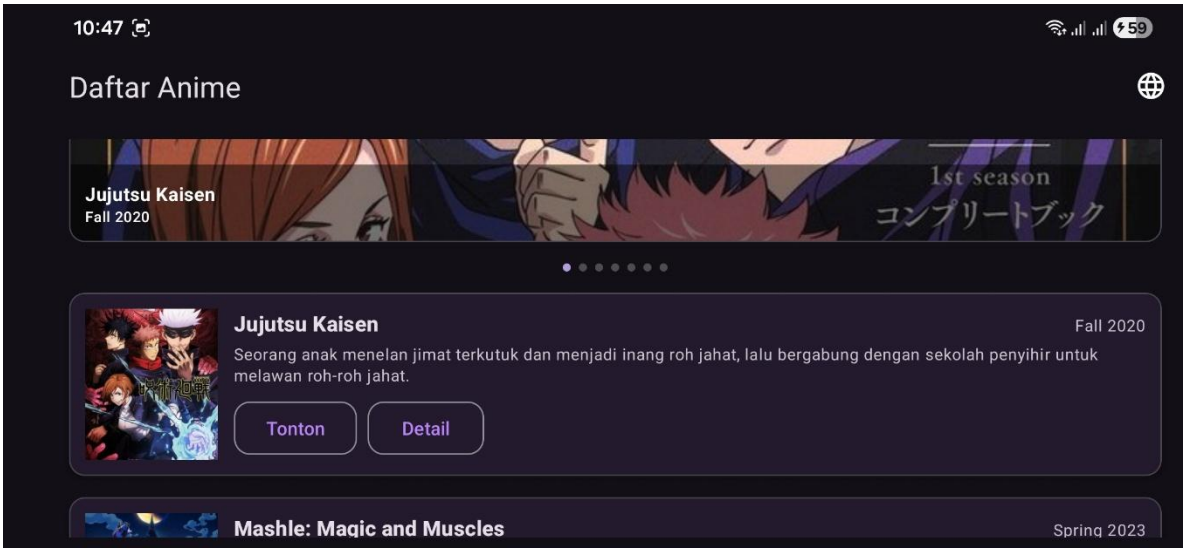
Gambar 14. Screenshot Tampilan Detail XML



Gambar 15. Screenshot Tampilan Pergantain Bahasa XML



Gambar 16. Screenshot Tampilan Setelah Ganti Bahasa XML



Gambar 17. Screenshot Tampilan Landscape XML

C. Pembahasan

MainActivity.kt (COMPOSE)

File ini adalah titik masuk utama aplikasi. Berisi tiga hal utama, yaitu setup awal activity, konfigurasi navigasi antar screen, dan tampilan layar utama (MainScreen).

Anime.kt (COMPOSE)

File ini adalah data class yang menjadi blueprint dari setiap objek anime. Semua data anime di aplikasi ini mengacu pada struktur yang didefinisikan di sini. Karena menggunakan data class, Kotlin otomatis men-generate fungsi seperti equals(), hashCode(), dan copy().

AnimeData.kt (COMPOSE)

File ini berfungsi sebagai sumber data statis (hardcoded). Berisi satu list bernama animeList yang menampung tujuh objek Anime. Karena menggunakan listOf(), data ini bersifat immutable (tidak bisa diubah).

AnimeCarousel.kt (COMPOSE)

File ini menampilkan carousel (pager horizontal) di bagian atas layar utama, lengkap dengan indikator titik (dot indicator) di bawahnya. Menggunakan HorizontalPager dari Compose Foundation.

AnimeList.kt (COMPOSE)

File ini adalah komponen kartu item list. Setiap anime di layar utama ditampilkan menggunakan komponen ini, berisi gambar, info, dan dua tombol aksi: Watch (buka Crunchyroll) dan Detail (navigasi ke halaman detail).

DetailScreen.kt (COMPOSE)

File ini adalah layar detail untuk setiap anime. Menerima parameter judul (title) dari navigasi, lalu mencari data anime yang cocok menggunakan fungsi find dari Kotlin Collection.

SettingScreen.kt (COMPOSE)

File ini menangani fitur ganti bahasa (lokalisasi) secara runtime. Menggunakan API LocaleManager untuk Android 13+ dan AppCompatDelegate untuk versi Android yang lebih lama, strategi dua jalur untuk kompatibilitas lintas versi.

AnimeViewModel.kt (COMPOSE)

File ini mengimplementasikan pola MVVM (Model-View-ViewModel). ViewModel berfungsi menyimpan dan mengelola state UI agar tetap bertahan saat konfigurasi berubah (misalnya layar diputar). Data diekspos lewat StateFlow sehingga bersifat reaktif.

Alur Program:

MainActivity adalah titik masuk. Saat dibuka, ia mengatur tema gelap lewat COMPOSETheme(darkTheme = true), lalu memanggil MyApp() yang menginisialisasi sistem navigasi dengan NavHost. Layar awal adalah MainScreen, yang menampilkan TopAppBar dengan ikon bahasa dan konten utama berupa LazyColumn.

Di dalam LazyColumn, ada dua section, pertama AnimeCarousel (carousel gambar dengan infinite scroll), lalu deretan AnimeList (kartu per anime). Setiap kartu punya dua tombol, tombol Watch membuka Crunchyroll lewat Intent, tombol Detail memanggil navController.navigate("details/{title}") untuk berpindah ke DetailScreen.

DetailScreen menerima judul anime sebagai argumen navigasi, mencari datanya di animeList menggunakan find {}, lalu merender detail lengkap. SettingScreen menangani ganti bahasa runtime dengan pendekatan dua jalur: LocaleManager untuk Android 13+, dan AppCompatDelegate untuk versi lebih lama.

Semua data bersumber dari animeList di AnimeData.kt, yang strukturnya didefinisikan oleh data class Anime di Anime.kt.

MainActivity.kt (XML)

Di versi XML, MainActivity sangat tipis, hanya bertugas meng-inflate layout dan menyetel content view. Semua logika navigasi dan UI diserahkan ke Fragment. Ini berbeda dengan versi Compose yang meletakkan NavHost di MainActivity.

Anime.kt (XML)

Sama persis dengan versi Compose, file ini adalah data class yang mendefinisikan struktur objek Anime. Semua field bertipe sama, termasuk description dan image yang bertipe Int karena menyimpan ID resource Android.

AnimeData.kt (XML)

Sama dengan versi Compose, berisi listOf() dengan tujuh objek Anime yang sudah di-hardcode. File ini menjadi sumber tunggal data di seluruh aplikasi XML.

HomeFragment.kt (XML)

Fragment ini adalah layar utama. Tugasnya menyiapkan Toolbar, lalu membangun RecyclerView yang menggabungkan carousel dan daftar anime menggunakan ConcatAdapter, teknik untuk menyatukan beberapa adapter menjadi satu list vertikal.

DetailFragment.kt (XML)

Fragment yang menampilkan detail lengkap satu anime. Menerima judul anime lewat Bundle (bukan parameter fungsi seperti di Compose), lalu mencari data dengan find, dan mengisi view menggunakan operator safe-call ?.let.

SettingFragment.kt (XML)

Fragment untuk mengganti bahasa aplikasi. Logikanya sama dengan versi Compose, tapi ada satu perbedaan penting, setelah memanggil setLocale(), versi XML secara eksplisit memanggil requireActivity().recreate() untuk memaksa Activity restart dan menerapkan bahasa baru.

AnimeViewModel.kt (XML)

Strukturnya identik dengan versi Compose, tetapi di sini ViewModel benar-benar digunakan oleh HomeFragment lewat delegate by viewModels(). Data diambil dengan viewModel.animeList.value.

AnimeAdapter.kt (XML)

Adapter ini menghubungkan data List<Anime> dengan tampilan item_anime.xml. Menggunakan pola ViewHolder agar view tidak di-inflate berulang kali (efisien). Tombol Detail diteruskan ke Fragment lewat lambda onDetailClick.

CarouselAdapter.kt (XML)

Adapter untuk ViewPager2 di dalam carousel. Teknik yang dipakai sama dengan versi Compose, getItemCount() mengembalikan Int.MAX_VALUE untuk efek infinite scroll, dan modulo digunakan saat bind data.

CarouselWrapperAdapter.kt (XML)

Adapter pembungkus yang mengemas ViewPager2 dan dot indicator menjadi satu item di dalam RecyclerView utama. getItemCount() mengembalikan 1 karena carousel hanya satu blok, tapi di dalamnya ViewPager2 bisa geser bebas.

activity_main.xml (xml)

Layout ini adalah container tunggal seluruh aplikasi. Hanya berisi satu elemen penting, FragmentContainerView yang menjadi wadah tempat semua Fragment ditampilkan dan diganti-ganti oleh Navigation Component.

fragment_home.xml (xml)

Layout ini sangat sederhana, hanya berisi Toolbar di atas dan RecyclerView yang mengisi sisa ruang. Dot indicator ada di sini tapi disetel visibility="gone" karena dot indicator **dikelola di dalam** item_carousel_wrapper.xml.

fragment_detail.xml (xml)

Layout detail dibungkus dalam ScrollView agar bisa digulir saat sinopsis panjang. Di dalamnya ada gambar besar 300dp, lalu teks judul, tahun, label sinopsis, isi sinopsis, dan tombol kembali.

fragment_setting.xml (xml)

Layout sederhana berbasis LinearLayout vertikal dengan padding 16dp. Berisi Toolbar di atas, label teks, dua tombol bahasa, dan tombol kembali.

item_anime.xml (xml)

Template tampilan untuk setiap baris anime di RecyclerView. Menggunakan MaterialCardView sebagai container dengan sudut melengkung dan bayangan, lalu isi berupa layout horizontal, gambar di kiri, info + tombol di kanan.

item_carousel.xml (xml)

Template satu slide di dalam ViewPager2. Gambar latar memenuhi card, dengan overlay gelap semi-transparan di bagian bawah untuk keterbacaan teks judul dan tahun tayang.

item_carousel_wrapper.xml (xml)

Layout untuk satu item RecyclerView yang berisi ViewPager2 (carousel bergulir) di atas, dan LinearLayout dot indicator di bawahnya. Keduanya terbungkus dalam satu LinearLayout vertikal.

Alur Program:

MainActivity hanya bertugas inflate activity_main.xml yang isinya cuma satu FragmentContainerView sebagai wadah navigasi. Semua logika ada di Fragment.

HomeFragment adalah layar utama, ia mengambil data dari AnimeViewModel via by viewModels(), lalu membangun RecyclerView dengan ConcatAdapter yang menggabungkan CarouselWrapperAdapter (berisi ViewPager2 + dot) di atas dan AnimeAdapter (kartu list anime) di bawahnya. Inilah teknik utama yang membedakan versi XML dari versi Compose.

CarouselWrapperAdapter adalah solusi arsitektur yang cukup kompleks: ia membungkus ViewPager2 sebagai satu item RecyclerView, lalu CarouselAdapter di dalamnya menggunakan trik Int.MAX_VALUE + modulo untuk infinite scroll. Dot indicator diisi secara programatik lewat addView().

Ketika tombol Detail diklik, AnimeAdapter memanggil lambda onDetailClick, yang diteruskan HomeFragment untuk mengirim Bundle berisi judul anime lalu navigate ke DetailFragment. DetailFragment membaca Bundle tersebut, mencari data dengan find{}, dan mengisi view lewat ?.let {}.

SOAL 2

Mengapa RecyclerView masih digunakan, padahal RecyclerView memiliki kode yang panjang dan bersifat boiler-plate, dibandingkan LazyColumn dengan kode yang lebih singkat?

A. Jawaban:

Walaupun RecyclerView memiliki kode yang lebih panjang dan bersifat boilerplate dibandingkan LazyColumn, RecyclerView masih tetap digunakan, beberapa hal yang menjadikannya alasan. Pertama, kontrol dan kustomisasi yang lebih tinggi. RecyclerView menyediakan fitur-fitur yang sangat fleksibel, beberapa diantaranya yaitu:

- ViewHolder Pattern yang wajib digunakan, membuat pengelolaan memori lebih efisien
- LayoutManager yang beragam seperti LinearLayoutManager, GridLayoutManager, serta StaggeredLayoutManager untuk berbagai kebutuhan tampilan
- ItemAnimator untuk animasi item yang lebih mudah dikontrol
- ItemDecorator untuk kustomisasi tampilan item secara dinamis

Kedua, ekosistem yang sudah matang. RecyclerView sudah digunakan bertahun-tahun dalam pengembangan Android berbasis View System, sehingga banyak proyek lama yang masih bergantung padanya dan belum berpindah ke Jetpack Compose.

Ketiga, perbedaan cara kerja. LazyColumn bekerja dengan cara membuang dan membuat ulang elemen UI sesuai kebutuhan, dengan optimasi di level kompilasi Kotlin. Sedangkan RecyclerView bekerja dengan cara mendaur ulang view yang sudah ada. Pada kasus daftar yang sangat kompleks dengan banyak tipe item, pendekatan RecyclerView memiliki performa yang lebih baik.

Keempat, konteks penggunaan yang berbeda. RecyclerView cocok untuk daftar kompleks dengan berbagai tipe item dan interaksi rumit. Sedangkan LazyColumn lebih ideal untuk daftar sederhana dalam proyek yang sepenuhnya menggunakan Jetpack Compose.

RecyclerView masih relevan terutama untuk proyek yang menggunakan View System dan membutuhkan kustomisasi tinggi. LazyColumn hadir bukan untuk menggantikan sepenuhnya, melainkan sebagai alternatif modern dalam ekosistem Jetpack Compose.

SOAL 3

Pada desain arsitektur kotlin, terdapat beberapa arsitektur dengan salah satunya adalah CLEAN ARCHITECTURE, menurut **pendapat kalian**, mengapa kita harus menerapkan arsitektur ini ketika membuat sebuah project?

A. Jawaban

Clean Architecture adalah suatu konsep arsitektur perangkat lunak yang bertujuan untuk menciptakan sistem perangkat lunak yang mudah dimengerti, mudah diuji, dan mudah diubah tanpa mengganggu bagian-bagian lain dari sistem tersebut.

Jadi, Clean Architectur membagi project menjadi beberapa layer, yaitu Domain Layer yang berada paling dalam, berisi kode-kode yang terkait langsung dengan bisnis logic atau domain. Lalu ada Use Case Layer, berisi kode-kode yang terkait dengan use case atau scenario penggunaan dari perangkat lunak. Lalu ada Interface Adapter Layer, berisi kode-kode yang terkait dengan antarmuka atau interface yang digunakan untuk berinteraksi dengan perangkat lunak. Terakhir ada Infrastructure Layer merupaka layer paling luar, berisi kode-kode yang terkait dengan infrastruktur atau lingkungan teknologi tertentu seperti database, jaringan, atau layanan pihak ketiga.

Menurut saya, alasan mengapa kita harus menerapkan arsitektur ini ketika membuat sebuah project adalah karena Clean Architecture memudahkan pengembangan aplikasi, di mana setiap layer dapat dikembangkan dan diuji secara terpisah tanpa mempengaruhi layer lainnya. Selain itu, Clean Architecture juga memungkinkan perubahan framework atau teknologi dengan mudah tanpa harus mempengaruhi bagian inti dari aplikasi, contohnya ketika ingin mengganti database dari MySQL ke MongoDB, kita cukup mengubah di Infrastructure Layer tanpa menyentuh lapisan inti.

Clean Architecture juga memudahkan pengujian karena unit testing dapat dilakukan pada setiap layer secara mandiri. Tidak hanya itu, arsitektur ini juga meningkatkan scalability karena setiap layer memiliki tanggung jawab yang jelas berdasarkan prinsip Single

Responsibility Principle (SRP), sehingga aplikasi mudah di-scaling tanpa mengganggu layer lainnya. Terakhir, Clean Architecture memperjelas struktur aplikasi sehingga memudahkan developer dalam proses debugging dan maintenance.

TAUTAN GITHUB

Link GitHub Modul 3